

Chunk 조합(Assemble Map) 구조 정리

Assets/Scripts/Editor/MapGenerator/AssembleChunk/

1. 전체 개요

기존에는 `ChunkMapGeneratorWindow.cs` 한 파일에 GUI와 두 생성기(선형/가지치기)가 전부 들어있었고, "Generate Chunk"(청크 자체를 새로 만드는) 모드도 같이 있었다. 지금은 **Generate Chunk 모드** (`ChunkAutoGenerator.cs`, `ChunkReachability.cs`)를 완전히 제거하고, 남은 Assemble Map 로직을 역할별로 4개 파일로 분리했다.

파일	역할
<code>ChunkMapGeneratorWindow.cs</code>	순수 GUI. <code>Open</code> / <code>OnEnable</code> / <code>OnGUI</code> / <code>DrawAssembleMapGUI</code> 만 담당하고, 버튼 클릭 시 아래 두 생성기를 호출만 한다.
<code>ChunkPlacementUtility.cs</code>	두 생성기가 공유하는 저수준 배치·연결 유틸리티.
<code>ChunkLinearGenerator.cs</code>	선형 생성기 — <code>Generate()</code>
<code>ChunkBranchingGenerator.cs</code>	가지치기 생성기 — <code>GenerateBranching()</code> (현재 메인으로 쓰는 방식)

세 알고리즘 파일은 모두 `internal static class` 이고, `using static MapSystem.Editor.ChunkPlacementUtility;` 로 공용 로직을 가져다 쓴다.

2. 공통 핵심 로직 — 연결 판정 (`ChunkPlacementUtility.cs`)

모든 생성기가 공유하는 원시 연산은 "청크 하나를 커서 위치에 연결해보기"다.

- `TryConnect(entry, cursor, ...)`: `entry.GetAllEntrances()` (N개 지점) 중 하나를 무작위 순서로 시도해서 입구로 삼고, 배치가 기존 박스들과 안 겹치면 나머지 진입점 중 무작위 하나를 출구(다음 커서)로 확정한다. 진입점이 1개뿐(막다른길)이면 그 청크는 연결 후보에서 제외된다.
- `TryConnectBranch(entry, cursor, ..., out unusedWorldPoints)`: 위와 동일하지만, 사용하지 않은 나머지 진입점들의 월드 좌표도 같이 반환한다 — 다방향(3개 이상) 청크를 만났을 때 그 나머지 지점들을 사이드 브랜치 시작점으로 쓰기 위함이다.
- `IsDeadEndEntry(entry)`: 모든 진입점 좌표가 동일하면 막다른길로 판단한다(입구=출구).
- 그 외 `PlacementFrame` (백트래킹 프레임 데이터), `RootName`, `RandomContentType`, `Overlaps`, `Shuffle<T>` 도 여기 있다.

3. 선형 생성기 — `ChunkLinearGenerator.Generate(db, steps, seed)`

- `PlacementFrame` 스택으로 한 스텝씩 진행한다: 매 스텝마다 `Transition/Content` 중 무작위로 원하는 타입을 정하고, DB에서 후보를 골라 `TryConnect` 를 시도한다.
- 막히면 그냥 건너뛰지 않고 **이전 스텝으로 되돌아가(step--)** 그 자리를 다른 청크로 재시도한다(백트래킹). 이전도 막히면 계속 더 앞으로 되돌아간다.
- `MAX_BACKTRACKS` (3000회)를 넘으면 그 지점까지만 배치하고 중단한다.

4. 가지치기 생성기 — `ChunkBranchingGenerator.GenerateBranching(db, contentChunkCount, seed)`

구조: **Start** → (**Content+Transition 반복**) → **End**, 다방향 청크에서 결가지가 자연스럽게 갈라진다. 5단계로 진행된다.

1. **Start 배치**: `EndLineRole.Start` 청크 하나를 원점에 연결한다.
2. **메인 경로**: `Generate()` 와 동일한 백트래킹 방식으로 `Content`가 `contentChunkCount` 개 놓일 때까지 진행한다. 다방향 청크를 만나면 안 쓴 진입점들을 `sideBranches` 대기열에 저장한다(`stepBudget` = 무작위 2~5).
3. **사이드 브랜치 처리**: 대기열을 하나씩 꺼내서 무작위 스텝만큼 진행 후, 반드시 `EndLineRole.Normal` 막다른 길로 마무리한다. 브랜치 도중 또 다방향 청크가 나오면 재귀적으로 대기열에 더 쌓인다(`branchGuard` 500 회 상한).
4. **End 배치 시도 + 확장(extend)**: 메인 경로 끝 커서(`mainEndCursor`)에 바로 End를 붙일 수 있는지 확인한다 (`canPlaceEndAt`). 안 되면 청크를 하나씩 더 끼워 넣어 커서를 밀어내면서(`extendHistory` 에 기록) 다시 시도한다 — 줄이지 않고 늘리는 방향으로만 접근한다.
5. **막히면 백트래킹**: `extend`가 막히면 최근 3개(`UNDO_ON_STUCK`)를 되돌리고, 그래도 안 되면 메인 스택 마지막 청크까지 되돌려서 근본적으로 경로를 바꿔본다. `MAX_EXTEND_ATTEMPTS` (300회)를 넘으면 포기한다.

5. 데이터 의존성

위 알고리즘은 아래 두 데이터 클래스에 의존한다 (`Assets/Scripts/Map/ChunkDatabase/` , 런타임에서도 접근 가능한 위치):

- `ChunkDatabase.cs` — `entries` 리스트 보관, `GetByType` / `GetByTypeAndDifficulty` 로 필터링
- `ChunkDatabaseEntry.cs` — `entrances` / `width` / `height` / `type` / `difficulty` / `endLineRole` / `prefab`

6. 알려진 이슈

@End 배치가 약 20~35% 확률로 실패하는 문제가 아직 남아있다 — 주로 막다른 가지에서 확장할 공간이 없을 때 발생한다. 2단계 백트래킹(브랜치 확장 이력 되돌리기 + 메인 시퀀스 되돌리기)을 추가하면 성공률이 오를 것으로 보이지만, 반복 실패 데이터를 더 모은 뒤 결정하기로 의도적으로 보류 중인 상태다.